

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ  
РОССИЙСКОЙ ФЕДЕРАЦИИ  
Федеральное государственное автономное  
образовательное учреждение высшего образования  
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»**

**Институт перспективной инженерии  
Департамент цифровых, робототехнических систем и электроники**

**Отчет по лабораторной работе №2  
по дисциплине «Искусственный интеллект и машинное обучение»**

Выполнил студент Ромащенко Данил группы ИВТ-б-о-24-1

Подпись студента \_\_\_\_\_  
Работа защищена « » \_\_\_\_\_ 20\_\_ г.  
Проверил Воронкин Р.А. \_\_\_\_\_  
(подпись)

Ставрополь 2026

**Тема:** «Основы работы с библиотекой NumPy».

**Цель работы:** исследовать базовые возможности библиотеки NumPy языка программирования Python.

Ход работы

Вариант 17

**Ссылка на репозиторий:** [https://github.com/I1N322/lab\\_2.git](https://github.com/I1N322/lab_2.git)

1) Был создан общедоступный репозиторий, в котором использована лицензия MIT и язык программирование Python.

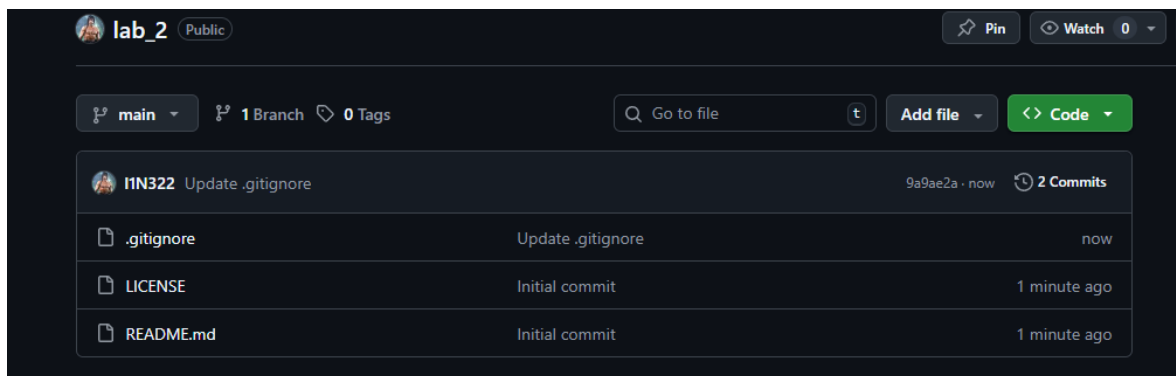


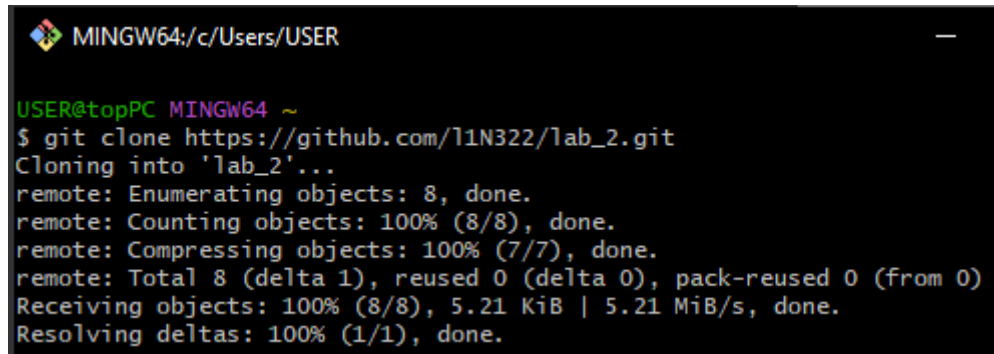
Рисунок 1. Созданный репозиторий

2) Был дополнен файл .gitignore необходимыми правилами.

```
209 # Covers JetBrains IDEs: IntelliJ, GoLand, RubyMine, PhpStorm, AppCode, PyCharm, CLion, Android Studio, WebStorm and Rider
210 # Reference: https://intellij-support.jetbrains.com/hc/en-us/articles/206544839
211
212 # User-specific stuff
213 .idea/**/workspace.xml
214 .idea/**/tasks.xml
215 .idea/**/usage.statistics.xml
216 .idea/**/dictionaries
217 .idea/**/shelf
218
219 # AWS User-specific
220 .idea/**/aws.xml
221
222 # Generated files
223 .idea/**/contentModel.xml
224
225 # Sensitive or high-churn files
226 .idea/**/dataSources/
227 .idea/**/dataSources.ids
228 .idea/**/dataSources.local.xml
229 .idea/**/sqlDataSources.xml
230 .idea/**/dynamic.xml
231 .idea/**/uiDesigner.xml
232 .idea/**/dbnavigator.xml
233
234 # Gradle
235 .idea/**/gradle.xml
236 .idea/**/libraries
237
238 # Gradle and Maven with auto-import
239 # When using Gradle or Maven with auto-import, you should exclude module files,
240 # since they will be recreated, and may cause churn. Uncomment if using
```

Рисунок 2. Добавленные правила в файл .gitignore

3) Было выполнено клонирование созданного репозитория на компьютер.



```
MINGW64:/c/Users/USER

USER@topPC MINGW64 ~
$ git clone https://github.com/11N322/lab_2.git
Cloning into 'lab_2'...
remote: Enumerating objects: 8, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (7/7), done.
remote: Total 8 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (8/8), 5.21 KiB | 5.21 MiB/s, done.
Resolving deltas: 100% (1/1), done.
```

Рисунок 3. Клонирование репозитория

4) Были выполнены задания по своему варианту (вариант 17).

Задание 1: Создайте массив NumPy размером  $3 \times 3$ , содержащий числа от 1 до 9. Умножьте все элементы массива на 2, а затем замените все элементы больше 10 на 0. Выведите итоговый массив.

```
[3]: import numpy as np

arr = np.arange(1, 10).reshape(3,3)
arr *= 2
arr[arr > 10] = 0
print(arr)

[[ 2  4  6]
 [ 8 10  0]
 [ 0  0  0]]
```

Рисунок 4. Задание 1

5) Задание 2: Создайте массив NumPy из 20 случайных целых чисел от 1 до 100. Найдите и выведите все элементы, которые делятся на 5 без остатка. Затем замените их на -1 и выведите обновленный массив.

```
[4]: import numpy as np

arr = np.random.randint(1, 101, size=20)
print(f"Исходный массив: {arr}")
divide_by_5 = arr[arr % 5 == 0]
print(f"Элементы массива, которые делятся на 5 без остатка: {divide_by_5}")
arr[arr % 5 == 0] = -1
print(f"Обновленный массив: {arr}")
```

Исходный массив: [ 66 31 7 64 97 6 72 61 17 64 87 35 28 93 100 20 12 60  
14 9]

Элементы массива, которые делятся на 5 без остатка: [ 35 100 20 60]

Обновленный массив: [66 31 7 64 97 6 72 61 17 64 87 -1 28 93 -1 -1 12 -1 14 9]

Рисунок 5. Задание 2

6) Задание 3: Создайте два массива NumPy размером  $1 \times 5$ , заполненные случайными числами от 0 до 50. Объедините эти массивы в один двумерный массив (по строкам). Разделите полученный массив на два массива, каждый из которых содержит 5 элементов. Выведите все промежуточные и итоговые результаты.

```
[1]: import numpy as np

arr1 = np.random.randint(0, 51, size=(1, 5))
arr2 = np.random.randint(0, 51, size=(1, 5))

print("Первый массив:", arr1)
print("Второй массив:", arr2)

combined = np.vstack((arr1, arr2))
print("Объединённый двумерный массив:\n", combined)

split1, split2 = np.vsplit(combined, 2)
print("Первый разделённый массив:", split1)
print("Второй разделённый массив:", split2)
```

Первый массив: [[ 3 20 5 29 13]]

Второй массив: [[17 2 21 12 14]]

Объединённый двумерный массив:

```
[[ 3 20  5 29 13]
 [17  2 21 12 14]]
```

Первый разделённый массив: [[ 3 20 5 29 13]]

Второй разделённый массив: [[17 2 21 12 14]]

Рисунок 6. Задание 3

7) Задание 4: Создайте массив из 50 чисел, равномерно распределенных от -10 до 10. Вычислите сумму всех элементов, сумму положительных элементов и сумму отрицательных элементов. Выведите результаты.

```
[4]: import numpy as np

arr = np.linspace(-10, 10, 50)
print(arr)

arr_sum = np.sum(arr)
positive_sum = np.sum(arr[arr > 0])
negative_sum = np.sum(arr[arr < 0])

print("\nСумма всех элементов:", arr_sum)
print("Сумма положительных элементов:", positive_sum)
print("Сумма отрицательных элементов:", negative_sum)
```

```
[-10.          -9.59183673 -9.18367347 -8.7755102  -8.36734694
 -7.95918367 -7.55102041 -7.14285714 -6.73469388 -6.32653061
 -5.91836735 -5.51020408 -5.10204082 -4.69387755 -4.28571429
 -3.87755102 -3.46938776 -3.06122449 -2.65306122 -2.24489796
 -1.83673469 -1.42857143 -1.02040816 -0.6122449  -0.20408163
  0.20408163  0.6122449  1.02040816  1.42857143  1.83673469
  2.24489796  2.65306122  3.06122449  3.46938776  3.87755102
  4.28571429  4.69387755  5.10204082  5.51020408  5.91836735
  6.32653061  6.73469388  7.14285714  7.55102041  7.95918367
  8.36734694  8.7755102  9.18367347  9.59183673 10.         ]
```

```
Сумма всех элементов: 7.105427357601002e-15
Сумма положительных элементов: 127.55102040816328
Сумма отрицательных элементов: -127.55102040816327
```

Рисунок 7. Задание 4

#### 8) Задание 5:

Создайте: Единичную матрицу размером  $4 \times 4$ . Диагональную матрицу размером  $4 \times 4$  с диагональными элементами  $[5, 10, 15, 20]$  (не использовать циклы). Найдите сумму всех элементов каждой из этих матриц и сравните результаты.

```
[1]: import numpy as np

idnt = np.eye(4)
diag = np.diag([5, 10, 15, 20])

sum_idnt = np.sum(idnt)
sum_diag = np.sum(diag)

print("Сумма элементов единичной матрицы:", sum_idnt)
print("Сумма элементов диагональной матрицы:", sum_diag)

if sum_idnt > sum_diag:
    print("Сумма элементов единичной матрицы больше")
elif sum_idnt < sum_diag:
    print("Сумма элементов диагональной матрицы больше")
else:
    print("Суммы равны")
```

```
Сумма элементов единичной матрицы: 4.0
Сумма элементов диагональной матрицы: 50
Сумма элементов диагональной матрицы больше
```

Рисунок 8. Задание 5

9) Задание 6: Создайте две квадратные матрицы NumPy размером  $3 \times 3$ , заполненные случайными целыми числами от 1 до 20. Вычислите и выведите: Их сумму, их разность, их поэлементное произведение

```
[1]: import numpy as np

A1 = np.random.randint(1, 21, size=(3, 3))
A2 = np.random.randint(1, 21, size=(3, 3))

print("Матрица 1:\n", A1)
print("Матрица 2:\n", A2)

print("Сумма:\n", A1 + A2)
print("Разность:\n", A1 - A2)
print("Поэлементное произведение:\n", A1 * A2)
```

Матрица 1:  
[[ 9 2 2]  
[ 6 14 6]  
[ 6 5 11]]

Матрица 2:  
[[ 7 8 15]  
[ 1 17 7]  
[ 2 4 1]]

Сумма:  
[[16 10 17]  
[ 7 31 13]  
[ 8 9 12]]

Разность:  
[[ 2 -6 -13]  
[ 5 -3 -1]  
[ 4 1 10]]

Поэлементное произведение:  
[[ 63 16 30]  
[ 6 238 42]  
[ 12 20 11]]

Рисунок 9. Задание 6

10) Задание 7:

Создайте две матрицы NumPy: Первую размером  $2 \times 3$ , заполненную случайными числами от 1 до 10. Вторую размером  $3 \times 2$ , заполненную случайными числами от 1 до 10. Выполните матричное умножение (@ или np.dot) и выведите результат.

```
[8]: import numpy as np

A = np.random.randint(1, 11, size=(2, 3))
B = np.random.randint(1, 11, size=(3, 2))
print("Первая матрица:\n", A)
print("Вторая матрица:\n", B)

result = np.dot(A, B)

print("Результат матричного умножения\n", result)

Первая матрица:
[[7 4 2]
 [4 1 4]]
Вторая матрица:
[[ 5  9]
 [ 8  8]
 [10  9]]
Результат матричного умножения
[[ 87 113]
 [ 68  80]]
```

Рисунок 10. Задание 7

11) Задание 8: Создайте случайную квадратную матрицу  $3 \times 3$ . Найдите и выведите: Определитель этой матрицы, Обратную матрицу (если существует, иначе выведите сообщение, что матрица вырождена). Используйте функции `np.linalg.det` и `np.linalg.inv`.

```
[5]: import numpy as np

A = np.random.rand(3, 3)
print("Случайная матрица:\n", A)

A_det = round(np.linalg.det(A), 3)
print("\nОпределитель матрицы", A_det)

if A_det == 0:
    print("\nМатрица вырождена!")
else:
    A_inv = np.linalg.inv(A)
    print("\nОбратная матрица:\n", A_inv)

Случайная матрица:
[[0.60251541 0.97619503 0.99440634]
 [0.53962837 0.36543589 0.73280125]
 [0.53085885 0.35565058 0.27344496]]

Определитель матрицы 0.137

Обратная матрица:
[[-1.17447534  0.63385494  2.57242277]
 [ 1.76473512 -2.6540573   0.69495779]
 [-0.01517103  2.22139592 -2.24087339]]
```

Рисунок 11. Задание 8

12) Задание 9: Создайте матрицу NumPy размером 4×4, содержащую случайные целые числа от 1 до 50. Выведите: Исходную матрицу, Транспонированную матрицу, След матрицы (сумму элементов на главной диагонали). Используйте `np.trace` для нахождения следа.

```
[2]: import numpy as np

A = np.random.randint(1, 51, size = (4, 4))
A_t = np.transpose(A)
A_trace = np.trace(A)

print("Исходная матрица:\n", A)
print("Транспонированная матрица:\n", A_t)
print("След матрицы:\n", A_trace)

Исходная матрица:
[[42  7 28  7]
 [32 18 36 23]
 [42 11 39 28]
 [17 49 47 50]]
Транспонированная матрица:
[[42 32 42 17]
 [ 7 18 11 49]
 [28 36 39 47]
 [ 7 23 28 50]]
След матрицы:
149
```

Рисунок 12. Задание 9

13) Задание 10: Решите систему линейных уравнений вида:

$$2x + 3y - z = 5$$

$$4x - y + 2z = 6$$

$$-3x + 5y + 4z = -2$$

Используйте матричное представление  $Ax = B$ , где  $A$  - матрица коэффициентов,  $x$  - вектор неизвестных,  $B$  - вектор правой части. Решите систему с помощью `np.linalg.solve` и выведите результат.

```
[1]: import numpy as np

A = np.array([[2, 3, -1],
              [4, -1, 2],
              [-3, 5, 4]])

B = np.array([5, 6, -2])

x = np.linalg.solve(A, B)
print("Решение системы (x, y, z):", x)

Решение системы (x, y, z): [1.63963964 0.57657658 0.00900901]
```

Рисунок 13. Задание 10

14) Индивидуальное задание: Решите индивидуальное задание. Задание предусматривает построение системы линейных уравнений. Решите полученную систему уравнений с использованием библиотеки NumPy. Для решения системы используйте метод Крамера и матричный метод. Сравните полученные результаты, с результатами, полученными с помощью `np.linalg.solve`.

Распределение электроэнергии. Город получает электроэнергию от трех электростанций. Первая станция дает в два раза больше энергии, чем вторая, а третья - на 200 МВт больше, чем вторая. Всего требуется 1500 МВт. Сколько электроэнергии должно поступать с каждой станции?

Пусть  $x_1$ ,  $x_2$ ,  $x_3$  - мощности (МВт) первой, второй, третьей станций соответственно.

$$x_1 = x_2 * 2$$

$$x_3 = x_2 + 200$$

$$x_1 + x_2 + x_3 = 1500$$

Получаем систему уравнений:

$$x_1 - x_2 * 2 = 0$$

$$x_3 - x_2 = 200$$

$$x_1 + x_2 + x_3 = 1500$$

```
[3]: import numpy as np

A = np.array([
    [1, -2, 0],
    [0, -1, 1],
    [1, 1, 1]
])
B = np.array([0, 200, 1500])

A_det = np.linalg.det(A)
if A_det == 0:
    print("Матрица вырождена, метод Крамера неприменим.")
else:
    x_cramer = []
    for i in range(3):
        Ai = A.copy()
        Ai[:, i] = B
        Ai_det = np.linalg.det(Ai)
        x_cramer.append(Ai_det / A_det)
    x_cramer = np.array(x_cramer)
    print("Метод Крамера:", x_cramer)

if A_det == 0:
    print("Матрица вырождена, обратная матрица не существует.")
else:
    A_inv = np.linalg.inv(A)
    x_inv = np.dot(A_inv, B)
    print("Матричный метод:", x_inv)

x_solve = np.linalg.solve(A, B)
print("np.linalg.solve:", x_solve)

print("\nВсе методы дают одинаковый результат (x1, x2, x3):")
print(f"x1 = {x_solve[0]}, x2 = {x_solve[1]}, x3 = {x_solve[2]}")

Метод Крамера: [650. 325. 525.]
Матричный метод: [650. 325. 525.]
np.linalg.solve: [650. 325. 525.]

Все методы дают одинаковый результат (x1, x2, x3):
x1 = 650.0, x2 = 325.0, x3 = 525.0
```

## Рисунок 14. Индивидуальное задание

### Ответы на контрольные вопросы:

1) NumPy (Numerical Python) — это фундаментальная библиотека для научных вычислений в Python. Она предоставляет:

- Многомерные массивы (ndarray) высокой производительности.
- Эффективные векторные и матричные операции.
- Широкий набор математических функций, работающих с массивами.
- Средства для линейной алгебры, преобразования Фурье, генерации случайных чисел и т.д.

– Основу для многих других библиотек (SciPy, pandas, scikit-learn, TensorFlow и др.).

2) Основная структура данных в NumPy - это многомерный массив, называемый ndarray. Это объект, который представляет собой таблицу данных (например, вектор или матрицу) и позволяет выполнять операции с ними с высокой эффективностью.

3) Доступ к частям многомерного массива осуществляется с помощью индексации и срезов.

4) NumPy предоставляет множество функций для расчёта статистических характеристик. Они могут применяться ко всему массиву или вдоль указанной оси.

| Имя метода    | Описание                                             |
|---------------|------------------------------------------------------|
| <b>argmax</b> | Индексы элементов с максимальным значением (по осям) |
| <b>argmin</b> | Индексы элементов с минимальным значением (по осям)  |
| <b>max</b>    | Максимальные значения элементов (по осям)            |
| <b>min</b>    | Минимальные значения элементов (по осям)             |
| <b>mean</b>   | Средние значения элементов (по осям)                 |
| <b>prod</b>   | Произведение всех элементов (по осям)                |
| <b>std</b>    | Стандартное отклонение (по осям)                     |
| <b>sum</b>    | Сумма всех элементов (по осям)                       |
| <b>var</b>    | Дисперсия (по осям)                                  |

5) Выборка — получение подмножества элементов. Способы: срезы, передача списков/массивов индексов, булева индексация.

6) Основные виды матриц и векторов:

– Вектор — одномерный массив (столбцовый или строчный, в NumPy различие не принципиально). `v = np.array([1,2,3])`

– Матрица — двумерный массив. `M = np.array([[1,2],[3,4]])`

– Квадратная матрица — число строк = число столбцов.

– Единичная матрица — единицы на главной диагонали. `I = np.eye(3)`

– Диагональная матрица — ненулевые элементы только на диагонали.  
`D = np.diag([1,2,3])`

– Симметричная матрица —  $A = A^T$ .

7) Транспонирование — операция, при которой строки матрицы становятся столбцами, а столбцы — строками.

8) Свойства:

– Дважды транспонированная матрица равна исходной матрице

– Транспонирование суммы матриц равно сумме транспонированных матриц

– Транспонирование произведения матриц равно произведению транспонированных матриц расставленных в обратном порядке

– Транспонирование произведения матрицы на число равно произведению этого числа на транспонированную матрицу

– Определители исходной и транспонированной матрицы совпадают

9) Средства NumPy для транспонирования: атрибут `.T` — наиболее удобный; функция `np.transpose(a, axes=None)` — позволяет переставлять оси многомерного массива.

10) Основные действия над матрицами:

– Сложение и вычитание

– Умножение на число

– Умножение матриц (скалярное, матричное, поэлементное)

– Транспонирование

– Вычисление определителя (для квадратных)

– Нахождение обратной матрицы

11) При умножении матрицы на число, все элементы матрицы умножаются на это число.

12) Свойства:

– Произведение единицы и любой заданной матрицы равно заданной матрице.

– Произведение нуля и любой матрицы равно нулевой матрице, размерность которой равна исходной матрицы.

- Произведение матрицы на сумму чисел равно сумме произведений матрицы на каждое из этих чисел.

- Произведение матрицы на произведение двух чисел равно произведению второго числа и заданной матрицы, умноженному на первое число.

- Произведение суммы матриц на число равно сумме произведений этих матриц на заданное число.

13) Складывать (вычитать) можно только матрицы одинаковой размерности - то есть матрицы, у которых совпадает количество столбцов и строк.

14) Свойства:

- Коммутативность сложения. От перестановки матриц их сумма не изменяется.

- Ассоциативность сложения. Результат сложения трех и более матриц не зависит от порядка, в котором эта операция будет выполняться.

- Для любой матрицы существует противоположная ей, такая, что их сумма является нулевой матрицей.

15) Средства NumPy для сложения и вычитания: операторы `+` и `-`; функции `np.add()`, `np.subtract()`; `np.sum()` для суммирования всех элементов или по оси.

16) Умножение матриц это уже более сложная операция, по сравнению с рассмотренными выше. Умножать можно только матрицы, отвечающие следующему требованию: количество столбцов первой матрицы должно быть равно числу строк второй матрицы.

17) Свойства:

- Ассоциативность умножения. Результат умножения матриц не зависит от порядка, в котором будет выполняться эта операция.

- Дистрибутивность умножения. Произведение матрицы на сумму матриц равно сумме произведений матриц.

– Умножение матриц в общем виде не коммутативно. Это означает, что для матриц не выполняется правило независимости произведения от перестановки множителей.

– Произведение заданной матрицы на единичную равно исходной матрице.

– Произведение заданной матрицы на нулевую матрицу равно нулевой матрице.

18) Средства NumPy для умножения матриц: `@` — оператор (начиная с Python 3.5); `np.dot()` — также работает для скалярного произведения векторов.

19) Определитель матрицы размера (n-го порядка) является одной из ее численных характеристик. Определитель матрицы  $A$  обозначается как  $|A|$  или  $\det(A)$ , его также называют детерминантом.

Свойства:

– Определитель матрицы остается неизменным при ее транспонировании.

– Если у матрицы есть строка или столбец, состоящие из нулей, то определитель такой матрицы равен нулю.

– При перестановке строк матрицы знак ее определителя меняется на противоположный.

– Если у матрицы есть две одинаковые строки, то ее определитель равен нулю.

– Если все элементы строки или столбца матрицы умножить на какое-то число, то и определитель будет умножен на это число.

– Если все элементы строки или столбца можно представить как сумму двух слагаемых, то определитель такой матрицы равен сумме определителей двух соответствующих матриц.

– Если к элементам одной строки прибавить элементы другой строки, умноженные на одно и то же число, то определитель матрицы не изменится.

– Если строка или столбец матрицы является линейной комбинацией других строк (столбцов), то определитель такой матрицы равен нулю.

– Если матрица содержит пропорциональные строки, то ее определитель равен нулю.

20) Средства NumPy для нахождения определителя: функция `np.linalg.det(A)`. Возвращает значение типа `float` (или комплексное).

21) Обратной матрицей  $A^{-1}$  матрицы  $A$  называют матрицу, удовлетворяющую следующему равенству:  $A * A^{-1} = A^{-1} \times A = E$ , где  $E$  - это единичная матрица.

Алгоритм нахождения:

- Вычислить определитель.
- Найти матрицу алгебраических дополнений.
- Транспонировать её — получить присоединённую матрицу.
- Разделить присоединённую матрицу на определитель.

22) Свойства:

- Обратная матрица обратной матрицы есть исходная матрица.
- Обратная матрица транспонированной матрицы равна транспонированной матрице от обратной матрицы.
- Обратная матрица произведения матриц равна произведению обратных матриц.

23) Функция `np.linalg.inv(A)` возвращает обратную матрицу, если она существует.

24) Каждая переменная выражается через определитель матрицы, где  $i$ -й столбец заменяется столбцом свободных членов. Формально это записывается как  $x_i = \det(A_i) / \det(A)$ , где  $A_i$  — матрица, полученная из  $A$  заменой  $i$ -го столбца на вектор  $b$ .

```
det_A = np.linalg.det(A)
```

```
if det_A == 0:
```

```
    print("Матрица вырождена, метод Крамера неприменим.")
```

```
else:
```

```
    x_cramer = []
```

```
    for i in range(3):
```

```
Ai = A.copy()
Ai[:, i] = B
det_Ai = np.linalg.det(Ai)
x_cramer.append(det_Ai / det_A)
x_cramer = np.array(x_cramer)
print("Метод Крамера:", x_cramer)
```

25) Матричный метод — это метод решения систем линейных уравнений, основанный на использовании обратной матрицы.

```
if det_A == 0:
    print("Матрица вырождена, обратная матрица не существует.")
else:
    A_inv = np.linalg.inv(A)
    x_inv = A_inv @ B
    print("Матричный метод:", x_inv)
```

**Вывод:** в ходе работы были исследованы базовые возможности библиотеки NumPy языка программирования Python.